



WELCOME





Analyzing Algorithm Efficiency : Time complexity

Analyzing Algorithm Efficiency

Agenda

Introduction

Time Complexity

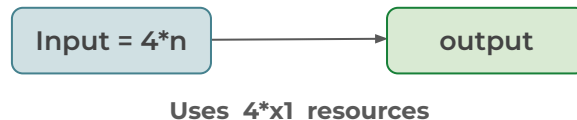
Asymptotic Notation

Introduction

Algorithm Efficiency refers to how effectively an algorithm performs in terms of the resources it uses as the input size increases.



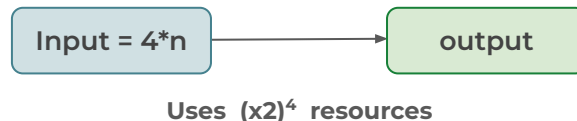
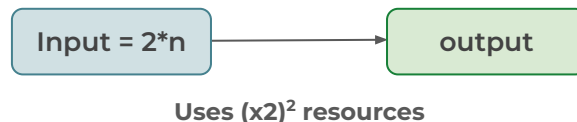
Consider Algorithm 1



Point to note:

On **increasing input** by an **additional value of n** ,
resources **consumed increase** by **additional value of $x1$** (linear).

Consider Algorithm 2



Point to note

On **increasing** input by **additional value n** ,
resources **consumed increase** by a **multiple of $x2$ (exponential)**.

Introduction

Algorithm 1

- Input = n → resource = $x1$
- Input = $2*n$ → resource = $2*x1$
- Input = $3*n$ → resource = $3*x1$
- Input = $4*n$ → resource = $4*x1$

So **resource consumption** is increasing **linearly** with input value.

Algorithm 2

- Input = n → resource = $x2$
- Input = $2*n$ → resource = $(x2)^2$
- Input = $3*n$ → resource = $(x2)^3$
- Input = $4*n$ → resource = $(x2)^4$

So **resource consumption** is increasing **exponentially** with input value.

Conclusion

From this, we can conclude that as the input size increases, the resource usage of Algorithm 1 grows slower compared to Algorithm 2.

Hence, Algorithm 1 is more efficient than Algorithm 2 in terms of resource usage for larger input values.



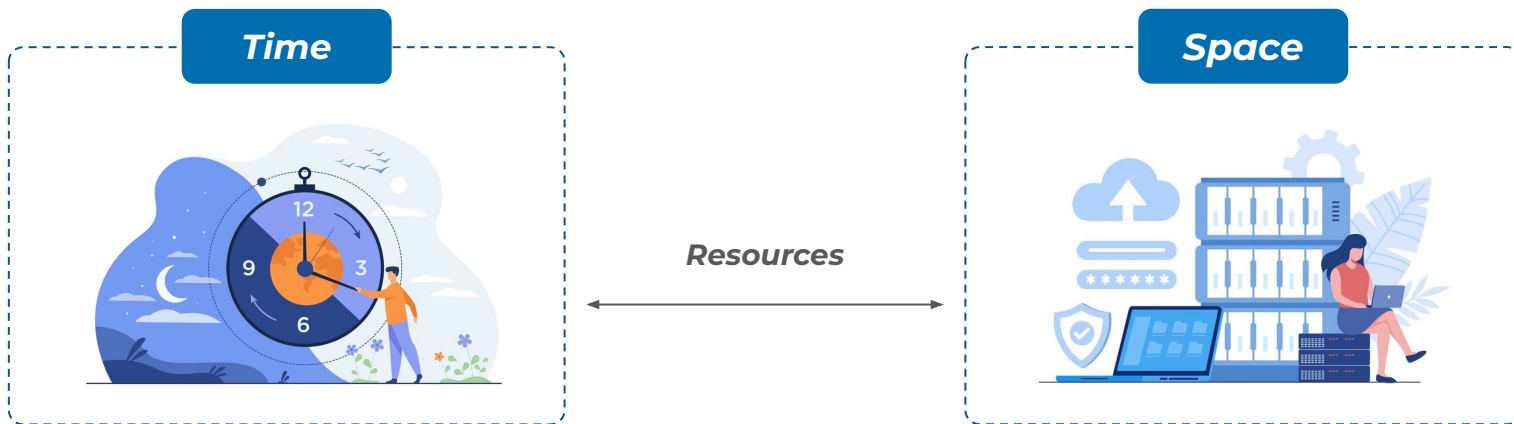
Note

For comparing efficiency, we do not need to compare the exact values of x_1 and x_2 here. Instead, we focus on how the resource consumption varies as the input size increases.

Analyzing Algorithm Efficiency

Algorithm Efficiency

When measuring Algorithm efficiency,
we consider two types of Resources.



Time - measures how long
the algorithm takes to
execute.

Space - measures how
much memory the
algorithm consumes.

Let's see both one by one

Time Complexity

Time Complexity

Time complexity analysis measures how the **speed of an algorithm changes as the input size increases** by counting the number of basic operations performed as a function of the input size.

Time Complexity

Code

_____ *Step 1*

_____ *Step 2*

_____ *Step 3*

_____ *Step 4*

_____ *Step 5*

_____ *Step 6*

⋮

_____ *Step n*

Time taken by the system to run a code:

Time taken = (# **steps**) * (**Time taken per step**)

Time Complexity

Code

_____ *Step 1*

_____ *Step 2*

_____ *Step 3*

_____ *Step 4*

_____ *Step 5*

_____ *Step 6*

⋮

_____ *Step n*

Time taken by the system to run a code:

Time taken = (# **steps**) * (Time taken per step)

Count of steps taken by the Algorithm
(Varies from **algorithm to algorithm**)

Time Complexity

Code

_____ *Step 1*

_____ *Step 2*

_____ *Step 3*

_____ *Step 4*

_____ *Step 5*

_____ *Step 6*

⋮

_____ *Step n*

Time taken by the system to run a code:

Time taken = (# steps) * **Time taken per step**

Varies from **device to device**
(Based on **device performance**).

Time Complexity

Time taken by the system to run a code:

$$\text{Time taken} = (\# \text{ steps}) * \text{Time taken per step}$$

Depends on the **efficiency**
of **algorithm**,

Ignore for algorithm
comparison as this is related
to **device performance**

Time Complexity

So, to **calculate time complexity**, we measure how the number of steps varies with the input size.



Let's understand with some examples:

Problem - 1

Code

```
for(int i = 1; i ≤ N; i++)  
{  
    cout<<i<<endl;  
}
```

*Given an input N ,
print the numbers from 1 to N .*

Problem - 1

Code

```
for(int i = 1; i ≤ N; i++)  
{  
    cout<<i<<endl;  
}
```

Initiation

.....➤ i = 1

Condition

.....➤ i ≤ N (**True**)

Print

.....➤ Output

Iteration: 1 (3 steps)

Problem - 1

Code

```
for(int i = 1; i ≤ N; i++)  
{  
    cout << i << endl;  
}
```

Updation

increased 'i'

Condition

$i \leq N$ (**True**)

Print

Output

Iteration: 2 to N (3 steps each)

Problem - 1

Code

```
for(int i = 1; i ≤ N; i++)  
{  
    cout<<i<<endl;  
}
```

Updation

→ $i = N+1$

Condition

→ $i \leq N$ (**False**)

Loop break (2 steps)

Problem - 1

- ✓ To calculate the time complexity of a **single 'for' loop**, the loop iterates **N** times from **i = 1** to **i = N** and then the loop performs **two** actions: **updating** i(to N+1) and checking the **condition**. Since the condition **fails** at **i = N+1**, the loop **terminates**.

$$\# \text{ Steps} = 3 * N + 2$$

The number of steps is given by **3N + 2**, indicating that the time taken by the algorithm varies linearly with the input size, with a multiplication factor of 3.

Problem - 2

Code

```
if(N%2==0) {  
    cout<<"even"<<endl;  
}  
else {  
    cout << "odd" << endl;  
}
```

Given an input N , print "even" if N is even and "odd " if N is odd.

Problem - 2

Code

```
if(N%2==0) {  
    cout<<"even"<<endl;  
}  
else {  
    cout << "odd" << endl;  
}
```

If N is even, the if block is executed; otherwise, the else block is executed.

However, the number of steps in the given code remains constant, i.e., 2, and is independent of the value of N.

Steps = 2

The number of steps is 2, indicating that the time taken by the algorithm is independent of the value of N.

Problem - 3

Code

```
for( int i=1; i ≤ N; i++ )  
{  
    for(int j=1; j ≤ N; j++)  
    {  
        cout<<"*";  
    }  
}
```

Given an input N , print the character '' N^2 times.*

Problem - 3

Code

```
for( int i=1; i ≤ N; i++ )  
{  
    for(int j=1; j ≤ N; j++)  
    {  
        cout<<"*";  
    }  
}
```

Initiation (i)

→ i = 1

Condition

→ 1 ≤ N (**True**)

Inner loop

→ Takes $3*N+2$ operations as calculated previously (since it is independent from outer loop).

Iteration : 1 ($3*N+4$ steps)

Problem - 3

Code

```
for( int i=1; i ≤ N; i++ )  
{  
    for(int j=1; j ≤ N; j++)  
    {  
        cout<<"*";  
    }  
}
```

Updation (i)



Increment i

Condition



i ≤ N (**True**)

Inner loop



Takes $3*N+2$ operations as calculated previously(since it is independent then outer loop)

Iteration : 2 to N ($3*N+4$ steps each)

Problem - 3

Code

```
for( int i=1; i ≤ N; i++ )  
{  
    for(int j=1;j ≤ N; j++)  
    {  
        cout<<"*";  
    }  
}
```

Updation (i)

→ i = N+1

Condition

→ i ≤ N (**False**)

Loop break (2 steps)

Problem - 3

- ✓ Summing up the steps described previously we get:

$$N*(3*N+4)+2 = 3*N*N + 4*N + 2$$

$$\# \text{ Steps} = 3*N*N + 4*N + 2$$



Comparison

Compare Different Algorithms

Different Algorithms in Time Complexity

	Algorithm 1	Algorithm 2	Algorithm 3
#Steps	$3N + 2$	2	$3N^2 + 4N + 2$

Let's **compare** how many **steps** each algorithm takes for **different inputs**.

Different Algorithms in Time Complexity

	Algorithm 1	Algorithm 2	Algorithm 3
#Steps	$3N + 2$	2	$3N^2 + 4N + 2$
$N = 1$	5	2	9

$$3N + 2$$
$$N = 1 \rightarrow 3(1) + 2$$
$$3 + 2 = 5$$

$$3N^2 + 4N + 2$$
$$N = 1 \rightarrow 3(1)^2 + 4(1) + 2$$
$$3 + 4 + 2 = 9$$

Different Algorithms in Time Complexity

	Algorithm 1	Algorithm 2	Algorithm 3
#Steps	$3N + 2$	2	$3N^2 + 4N + 2$
N = 1	5	2	9
N = 5	17	2	97
N = 100	302	2	30402
N = 100000	300002	2	3000 Cr+4 Lakh+ 2

When comparing 2 and 3000 Cr, 2 can be ignored

Different Algorithms in Time Complexity

	Algorithm 1	Algorithm 2	Algorithm 3
#Steps	$3N + 2$	2	$3N^2 + 4N + 2$
N = 1	5	2	9
N = 5	17	2	97
N = 100	302	2	30402
N = 100000	300002	2	3000 Cr + 4 Lakh + 2

Approximately 300000.

4 Lakh and 3000 Cr are also incomparable

Different Algorithms in Time Complexity

	Algorithm 1	Algorithm 2	Algorithm 3
#Steps	$3N + 2$	2	$3N^2 + 4N + 2$
N = 1	5	2	9
N = 5	17	2	97
N = 100	302	2	30402
N = 100000	300000	2	3000 Cr

This will take
medium time.

Less time

This task involves a **significant number of steps**, which means that it will require **maximum time to complete.**



Asymptotic Notation

Learning about Asymptotic Notation

Big O Notation

Asymptotic notations are mathematical tools that describe an algorithm's efficiency as the input size grows. They simplify analysis by focusing on dominant growth rates rather than exact runtimes. Here are the main types:

- Big O (O): Upper bound (worst-case).
- Big Omega (Ω): Lower bound.
- Big Theta (Θ): Tight bound (both upper and lower).

Asymptotic Notations

01 Big O Notation

02 Omega Notation

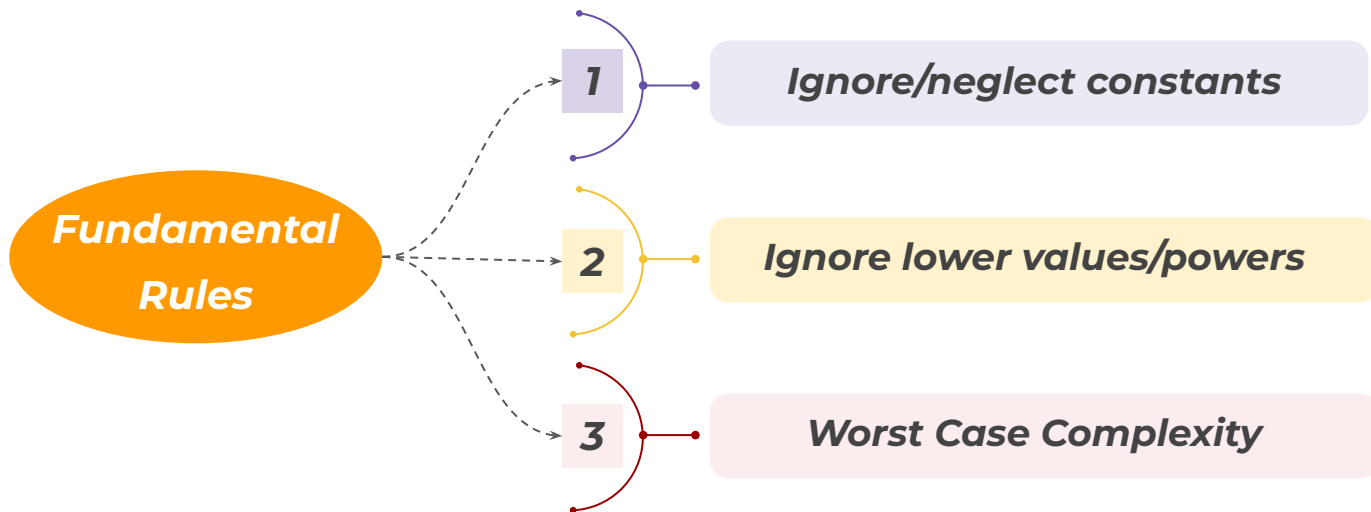
03 Theta Notation

Big O Notation

Big O Notation

- Big O notation is a mathematical representation used to describe the upper bound or worst-case scenario of an algorithm's time or space complexity.
- It provides a simplified way to express how the performance of an algorithm scales with input size.

Fundamental Rules

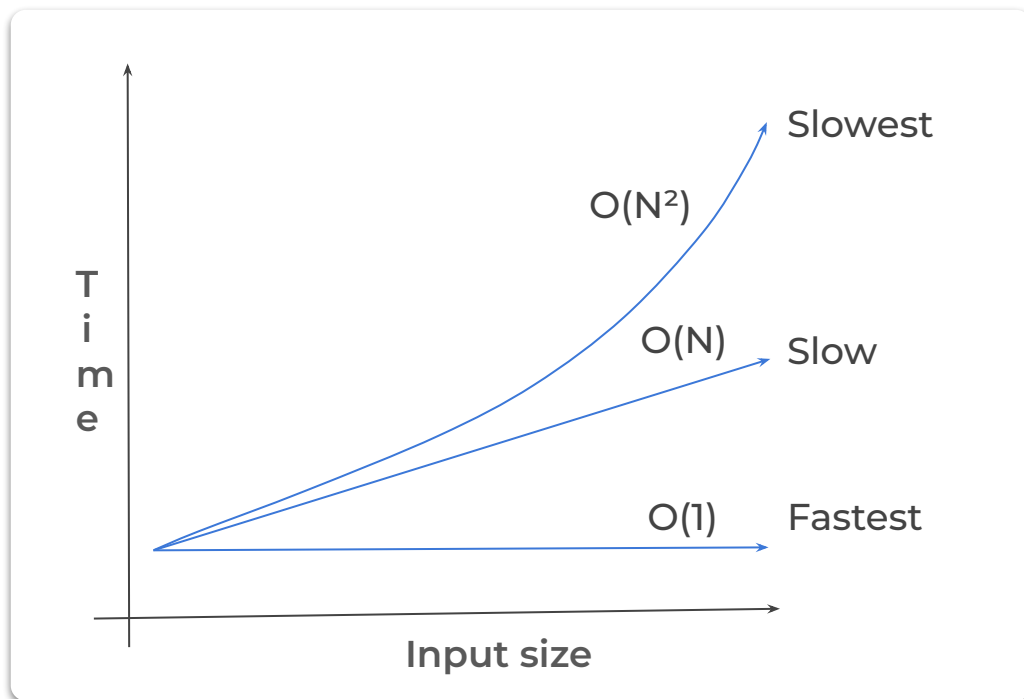


Comparison of Different Algorithms

	Algorithm 1	Algorithm 2	Algorithm 3
#Steps	$3N + 2$	2	$3N^2 + 5N + 2$
	↑ $O(N)$	↑ $O(1)$	↑ $O(N^2)$

We can **represent** the speed of any algorithm using **big O notation**.

Big O Notation



Big Ω (Omega) Notation

Big Ω (Omega) Notation is a mathematical representation used to describe the **lower bound** of an algorithm's **time** or **space** complexity.

It provides a simplified way to express the **minimum performance** of an **algorithm** as the **input size grows**.

Big Θ (Theta) Notation

Big Θ (Theta) Notation is a mathematical representation used to describe a **tight bound** on an algorithm's **time** or **space** complexity (both **upper** and **lower bounds**).

It provides a simplified way to express how the **performance of an algorithm scales** with **input size** from **both above** and **below**.

Key Takeaways

- ✓ **Algorithm efficiency** is about how resources like time and space grow as input increases.
- ✓ **Time Complexity** – Measures how the number of steps in an algorithm grows with input size (e.g., $O(1)$, $O(N)$, $O(N^2)$).
- ✓ **Asymptotic Notation** – Big O (worst case), Big Ω (best case), and Big Θ (average/tight bound) help describe algorithm efficiency.





THANK YOU





ALL THE BEST

